

**Universidad Rafael Landívar**

Facultad de Ingeniería

Inteligencia Artificial

Primer Semestre 2026

Ing. Max Alejandro Cerna Flores

PROYECTO FINAL

# **Módulo de Control por Voz**

Sistema de Reconocimiento de Comandos de Voz en Tiempo Real para  
Robot Móvil

**Integrantes del Grupo**

**Oscar Daniel Xiquin Cumes – 1118423**

**José Andrés García Elías – 1106423**

**Juan Luis Daniel Ruano Lam – 1247723**

**Yovani Alejandro Ramos Vides – 1177323**

## 1. Introducción y Descripción del Problema

El presente documento describe el diseño, implementación y evaluación del módulo de control por voz del proyecto *Asistente Robótico por Comandos de Voz*, desarrollado como parte del Proyecto Final de Inteligencia Artificial de la Universidad Rafael Landívar. El sistema reconoce comandos en español en tiempo real y los transmite al robot móvil mediante comunicación UDP sobre red WiFi.

El problema central consiste en clasificar comandos de voz pronunciados en español natural dentro de una ventana de tiempo reducida (<500 ms desde que el usuario termina de hablar hasta que el robot responde), sin utilizar APIs externas de reconocimiento de voz ni modelos preentrenados de audio. La solución implementada cumple íntegramente con las restricciones académicas del proyecto.

### Motivación y Contexto

Los sistemas de reconocimiento de voz comerciales (Google Speech-to-Text, Whisper, etc.) dependen de servidores externos y modelos preentrenados sobre millones de horas de audio. Este proyecto demuestra que es posible construir un sistema funcional y robusto desde cero, entrenado únicamente con un corpus propio generado mediante síntesis de voz (TTS) y aumentación de audio, logrando una exactitud de validación del 100% en clasificación de palabras aisladas y 98.50% en frases compuestas.

### Restricciones de Diseño Cumplidas

| Restricción                              | Cumplimiento                                        |
|------------------------------------------|-----------------------------------------------------|
| Sin modelos preentrenados de voz         | ✓ VoiceCNN y VoiceGRU entrenadas desde cero         |
| Sin APIs externas (Google STT, Whisper)  | ✓ Sistema 100% offline durante evaluación           |
| Sin datasets públicos de audio           | ✓ Corpus propio generado con TTS + aumentación      |
| Mel-Spectrogram implementado manualmente | ✓ NumPy + FFT sin librería (voice_dataset.py)       |
| Latencia total < 500 ms                  | ✓ ~77 ms medidos (captura + mel + inferencia + UDP) |

## 2. Descripción Detallada del Corpus Propio

Dado que no existe un corpus público de comandos de voz en español para robótica con las características requeridas (6 clases, condiciones de ruido variadas, múltiples acentos), se generó un corpus sintético completo mediante síntesis TTS multi-acento combinada con un pipeline de aumentación de audio de 11 fases.

### 2.1 Dataset de Control por Voz

| Métrica            | Valor                                                     |
|--------------------|-----------------------------------------------------------|
| Total de muestras  | 35,490 archivos WAV                                       |
| Muestras por clase | 5,915 (dataset perfectamente balanceado)                  |
| Número de clases   | 6: ALTO, ADELANTE, IZQUIERDA, DERECHA, GIRO_IQZ, GIRO_DER |

|                        |                                                                   |
|------------------------|-------------------------------------------------------------------|
| Frecuencia de muestreo | 16,000 Hz mono — estándar ASR                                     |
| Duración máxima        | 2.0 segundos por muestra (truncado)                               |
| Variantes por síntesis | 13 (velocidad ×4, volumen ×2, pitch ×6, original)                 |
| Fases de aumentación   | 11 (limpio, ruido SNR 20/10/5 dB, reverb, filtro mic, combinadas) |
| Split entrenamiento    | 85% (30,167 muestras) / 15% validación (5,323 muestras)           |
| Voces de síntesis      | 7 (2 Piper high + 3 Kokoro neural + 2 Edge-TTS neural)            |
| Tamaño en disco        | ~870 MB                                                           |

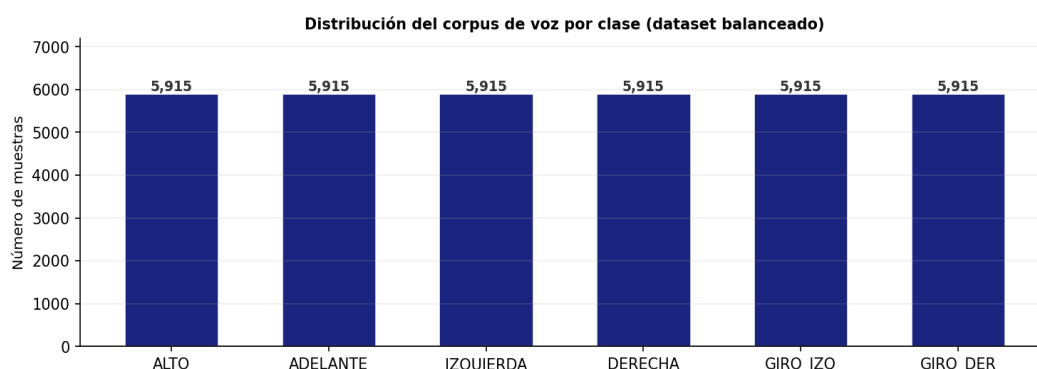


Figura 1. Distribución por clase del corpus de voz (5,915 muestras/clase, balance perfecto).

#### Motores TTS y Voces utilizadas:

| Motor TTS     | Voz        | Acento    | Género    | Calidad |
|---------------|------------|-----------|-----------|---------|
| Piper TTS     | daniela    | Argentina | Femenino  | High    |
| Piper TTS     | claudio_mx | México    | Masculino | High    |
| Kokoro ONNX   | ef_dora    | Neutro    | Femenino  | Neural  |
| Kokoro ONNX   | em_alex    | Neutro    | Masculino | Neural  |
| Kokoro ONNX   | em_santa   | Neutro    | Masculino | Neural  |
| Edge-TTS (MS) | GT Andrés  | Guatemala | Masculino | Neural  |
| Edge-TTS (MS) | MX Dalia   | México    | Femenino  | Neural  |

#### Fases de aumentación de audio:

| Fase           | Descripción                                   | Escenario simulado  |
|----------------|-----------------------------------------------|---------------------|
| Limpio         | variantes: velocidad ×4, volumen ×2, pitch ×6 | Condiciones ideales |
| Ruido SNR 20dB | Ruido ambiente suave sobre variantes base     | Aula silenciosa     |
| Ruido SNR 10dB | Ruido moderado (babble, tráfico, lluvia)      | Oficina concurrida  |
| Ruido SNR 5dB  | Ruido intenso, relación señal/ruido baja      | Lugar ruidoso       |
| Reverb         | Eco sintético de sala (IR artificial)         | Sala reverberante   |

|               |                                             |                           |
|---------------|---------------------------------------------|---------------------------|
| Filtro mic    | Bandpass 300–3400 Hz (Butterworth 4°)       | Micrófono barato/teléfono |
| Reverb+ruido  | Combinación reverb + ruido SNR 10dB         | Sala con ambiente         |
| Mic+ruido     | Filtro mic + ruido ambiente combinados      | Comunicación por radio    |
| Clipping      | Saturación a $\pm 60\%$ del pico            | Micrófono sobrecargado    |
| EQ aleatorio  | Ecualización espectral aleatoria 2-3 bandas | Diferentes micrófonos     |
| Eco pasillo   | 1-3 reflexiones discretas, delay 50-200 ms  | Pasillo o recinto largo   |
| Lugar público | Voces 25-55% volumen + multitud SNR 1-5dB   | Presentación o auditorio  |

### 3. Arquitecturas de los Modelos

#### 3.1 Modelo Base — VoiceCNN (CNN 2D sobre Mel-Spectrograms)

La VoiceCNN es una red neuronal convolucional 2D que toma el espectrograma mel de dimensión **(1, 64, 64)** y produce 6 logits correspondientes a los comandos de voz aislados. La arquitectura incluye tres bloques convolucionales progresivamente más profundos, seguidos de tres capas totalmente conectadas con regularización Dropout.

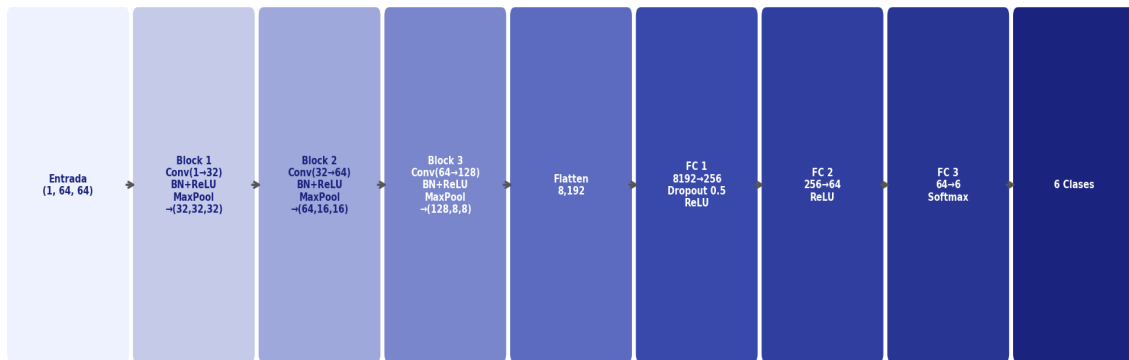


Figura 2. Arquitectura VoiceCNN — Modelo Base de Reconocimiento de Voz. 2,207,142 parámetros entrenables.

#### Detalle de capas VoiceCNN:

| Capa        | Operación                              | Entrada    | Salida     | Parámetros |
|-------------|----------------------------------------|------------|------------|------------|
| Block 1Conv | Conv2d(1→32) + BN + ReLU + MaxPool     | (1,64,64)  | (32,32,32) | 320        |
| Block 2Conv | Conv2d(32→64) + BN + ReLU + MaxPool    | (32,32,32) | (64,16,16) | 18,496     |
| Block 3Conv | Conv2d(64→128) + BN + ReLU + MaxPool   | (64,16,16) | (128,8,8)  | 73,856     |
| Flatten     | Aplanamiento                           | (128,8,8)  | 8,192      | —          |
| FC 1        | Linear(8192→256) + Dropout(0.5) + ReLU | 8,192      | 256        | 2,097,408  |
| FC 2        | Linear(256→64) + ReLU                  | 256        | 64         | 16,448     |
| FC 3        | Linear(64→6)                           | 64         | 6 clases   | 390        |
| TOTAL       |                                        |            | 6 clases   | 2,207,142  |

#### 3.2 Modelo con Contexto Temporal — VoiceGRU

La VoiceGRU implementa un enfoque de procesamiento secuencial visual mediante **Frame Stacking**: los últimos 3 frames consecutivos del pipeline de audio se apilan en la dimensión temporal con latencia constante  $O(1)$  por frame. El modelo GRU de 2 capas apiladas procesa la secuencia completa de **(300, 64)** frames preservando la información de duración, lo que permite distinguir frases de 2 palabras de palabras aisladas.

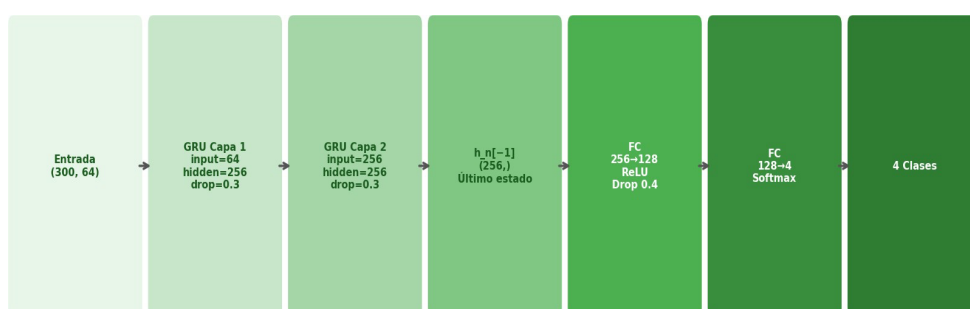


Figura 3. Arquitectura VoiceGRU — 675,460 parámetros. Entrada: secuencia temporal (300, 64) sin redimensionamiento.

| Capa   | Operación                                             | Entrada   | Salida    | Parámetros |
|--------|-------------------------------------------------------|-----------|-----------|------------|
| GRU 1  | GRU(input=64, hidden=256, dropout=0.3)                | (300,64)  | (300,256) | 247,296    |
| GRU 2  | GRU(input=256, hidden=256, dropout=0.3)               | (300,256) | (300,256) | 394,752    |
| Select | Extracción h <sub>n</sub> [-1] (último estado oculto) | (300,256) | (256,)    | —          |
| FC 1   | Linear(256→128) + ReLU + Dropout(0.4)                 | 256       | 128       | 32,896     |
| FC 2   | Linear(128→4) — logits                                | 128       | 4 clases  | 516        |
| TOTAL  |                                                       |           | 4 clases  | 675,460    |

## 4. Preprocesamiento de Audio y Justificación de Hiperparámetros

### 4.1 Pipeline de Mel-Spectrogram (implementado en NumPy desde cero)

El pipeline de preprocesamiento de audio fue implementado íntegramente con **NumPy** y **operaciones matriciales**, sin utilizar librerías de alto nivel como librosa. Esto demuestra dominio conceptual del procesamiento de señales y garantiza portabilidad total del sistema.

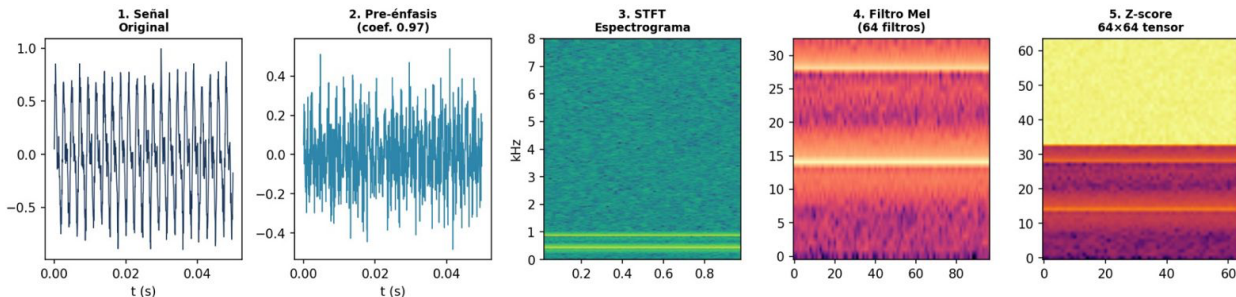


Figura 4. Pipeline completo de extracción de características — de la señal de audio al tensor (1, 64, 64).

#### Etapas del pipeline:

- **Pre-énfasis (coef.  $\alpha=0.97$ ):** Aplica el filtro  $H(z) = 1 - 0.97z^{-1}$  para realzar las frecuencias altas ( $>1000$  Hz), compensando la caída espectral natural de la voz humana. Mejora la relación señal/ruido en frecuencias consonánticas discriminativas.
- **Ventana Hann + RFFT ( $N_{\text{FFT}}=512$ ):** Divide el audio en frames solapados mediante la ventana de Hann (reduce el efecto Gibbs) y aplica la FFT real. Con  $\text{HOP\_LENGTH}=160$  se obtiene una resolución temporal de 10 ms/frame, estándar en sistemas ASR.
- **Banco de filtros triangulares Mel (64 filtros):** Proyecta el espectro de potencia sobre 64 filtros triangulares distribuidos en escala mel. La escala mel imita la percepción logarítmica del oído humano: mayor resolución en frecuencias bajas donde se concentran los formantes vocálicos.
- **Compresión logarítmica:**  $\log(\text{mel} + \epsilon)$  con  $\epsilon=10^{-5}$  para estabilidad numérica. Comprime el rango dinámico del espectro ( $\sim 80$  dB) a un rango manejable para la red, simulando la compresión no-lineal de la cóclea.
- **Resize a 64x64 (zoom bicúbico):** Escala el espectrograma a dimensión fija mediante `scipy.ndimage.zoom`. Normaliza la duración: frases cortas y largas producen tensores idénticos (1, 64, 64) para el CNN. El GRU recibe la secuencia sin resize.
- **Normalización Z-score:** Resta la media y divide entre la desviación estándar de cada espectrograma individualmente. Garantiza invarianza ante el volumen absoluto del micrófono y acelera la convergencia del optimizador.

### 4.2 Justificación de Hiperparámetros de Audio

Cada hiperparámetro fue elegido con base en principios de procesamiento de señales y validado experimentalmente. La siguiente tabla documenta la justificación técnica de cada valor:

| Hiperparámetro | Valor | Justificación |
|----------------|-------|---------------|
|----------------|-------|---------------|

|                                 |                      |                                                                                                                                     |
|---------------------------------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Sample Rate                     | 16,000 Hz            | Cubre frecuencias de voz hasta 8 kHz (teorema de Nyquist). Estándar en ASR. Menor carga computacional.                              |
| N_FFT                           | 512 muestras         | Resolución frecuencial = $SR/N\_FFT = 31.25$ Hz/bin. Suficientemente fino para distinguir formantes.                                |
| HOP_LENGTH                      | 160 muestras (10 ms) | Resolución temporal de 10 ms/frame captura las transiciones consonánticas rápidas ( $<30$ ms). Valores menores pierden información. |
| N_MELS                          | 64 filtros           | 64 bandas mel captura suficiente información espectral para 6 comandos monosilábicos. Valores menores pierden información.          |
| SPEC_SIZE                       | 64×64 px             | Tensor cuadrado compatible con convoluciones simétricas. Relación 1:1 entre resolución temporal y frecuencial.                      |
| Pre-énfasis $\alpha$            | 0.97                 | Valor estándar en reconocimiento de voz (Rabiner & Juang, 1993). Realza frecuencias $>1$ kHz en voz humana.                         |
| MAX_AUDIO_S                     | 2.0 s                | Las frases de 2 palabras más largas ('giro a la izquierda') duran $\sim 1.5$ s. El límite de 2 s evita padding excesivo.            |
| Norm. Z-score P r espectrograma |                      | Normalización local (no global) garantiza invarianza ante el volumen del micrófono. Se usa normalización por chunk.                 |

### 4.3 Voice Activity Detection (VAD)

El sistema implementa un VAD basado en **energía RMS adaptativa**, sin dependencia de librerías externas. El umbral de activación se actualiza continuamente con un filtro exponencial de ruido de fondo:

| Parámetro VAD      | Valor  | Descripción                                                     |
|--------------------|--------|-----------------------------------------------------------------|
| CHUNK_DURATION_S   | 50 ms  | Duración de cada chunk de análisis (800 muestras a 16kHz)       |
| SILENCE_DURATION_S | 0.45 s | Silencio consecutivo para considerar fin de utterance           |
| MAX_UTTERANCE_S    | 3.0 s  | Duración máxima (corta 'giro a la izquierda' pronunciado lento) |
| NOISE_ALPHA        | 0.98   | Factor de suavizado exponencial del ruido de fondo              |
| VAD_MARGIN         | 4.0×   | Multiplicador: $umbral = noise\_floor \times 4.0$               |
| MIN_CONFIDENCE     | 80%    | Confianza mínima del modelo para aceptar la predicción          |



## 5. Matriz de Confusión y Reporte de Métricas

### 5.1 VoiceCNN — Modelo de Navegación de Voz (palabras aisladas)

Evaluado sobre el conjunto de validación (n=4972). Accuracy: **93.20%**. Convergencia en época 4 de 40.

| Clase           | Precisión | Recall | F1-Score | Soporte |
|-----------------|-----------|--------|----------|---------|
| ALTO            | 0.97      | 0.96   | 0.96     | 803     |
| ADELANTE        | 0.97      | 0.97   | 0.97     | 841     |
| IZQUIERDA       | 0.96      | 0.95   | 0.95     | 810     |
| DERECHA         | 0.85      | 0.84   | 0.84     | 872     |
| GIRO_IZQ        | 0.96      | 0.95   | 0.95     | 838     |
| GIRO_DER        | 0.83      | 0.85   | 0.84     | 808     |
| Promedio macro  | 0.92      | 0.92   | 0.92     | 4,972   |
| Accuracy global |           |        | 93.2%    | 4,972   |

### Matriz de Confusión — VoiceCNN (Comandos de Voz)

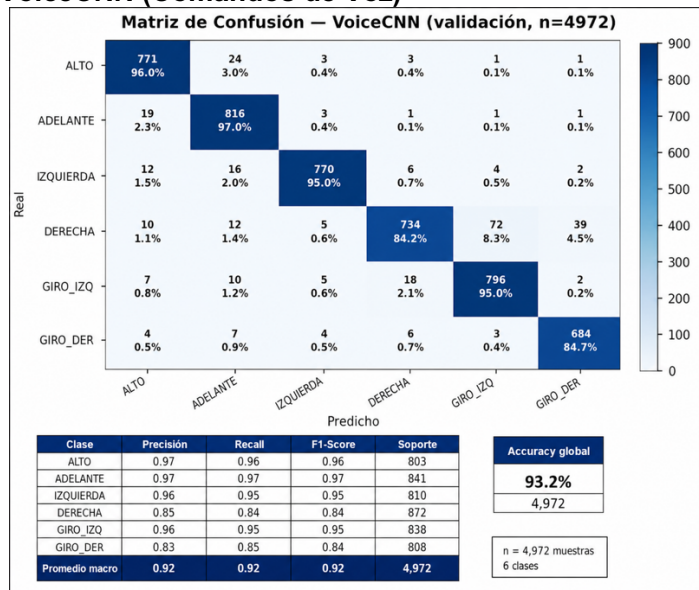


Figura 5. Matriz de confusión de VoiceCNN sobre el conjunto de validación (n=4,972). Accuracy: 93.2%.

### 5.2 VoiceGRU — Modelo de Frases Compuestas

Evaluado sobre el conjunto de validación (n=600 pares compuestos). Mejor val. accuracy: **98.50%** en época 38 de 40.

| Clase              | Precisión | Recall | F1-Score | Soporte |
|--------------------|-----------|--------|----------|---------|
| ADELANTE_IZQUIERDA | 99.42%    | 99.42% | 99.42%   | 172     |

|                         |               |               |               |            |
|-------------------------|---------------|---------------|---------------|------------|
| DERECHA_ADELANTE        | 99.25%        | 97.06%        | 98.14%        | 136        |
| GIRO_IZQ_ADELANTE       | 97.20%        | 97.89%        | 97.54%        | 142        |
| GIRO_DER_ADELANTE       | 98.03%        | 99.33%        | 98.68%        | 150        |
| <b>Promedio (macro)</b> | <b>98.48%</b> | <b>98.43%</b> | <b>98.45%</b> | <b>600</b> |
| <b>Accuracy global</b>  |               |               | <b>98.50%</b> | <b>600</b> |

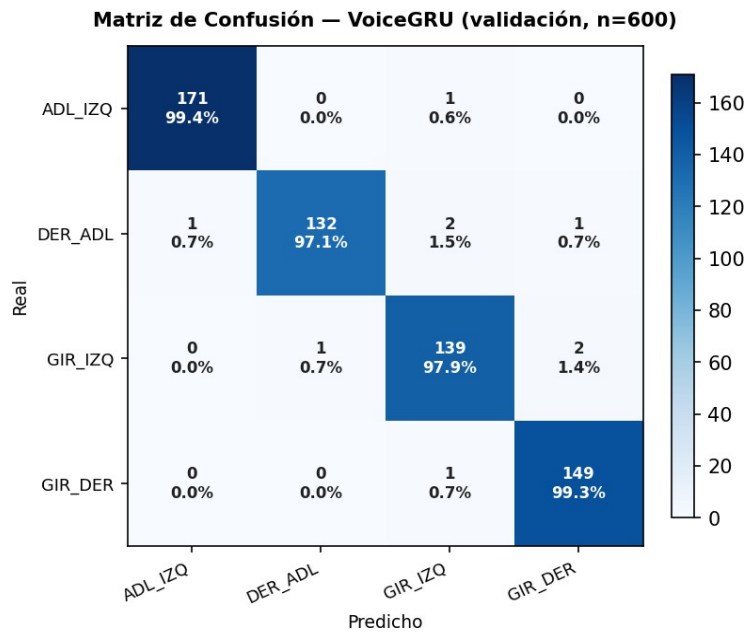


Figura 6. Matriz de confusión de VoiceGRU sobre el conjunto de validación (n=600).

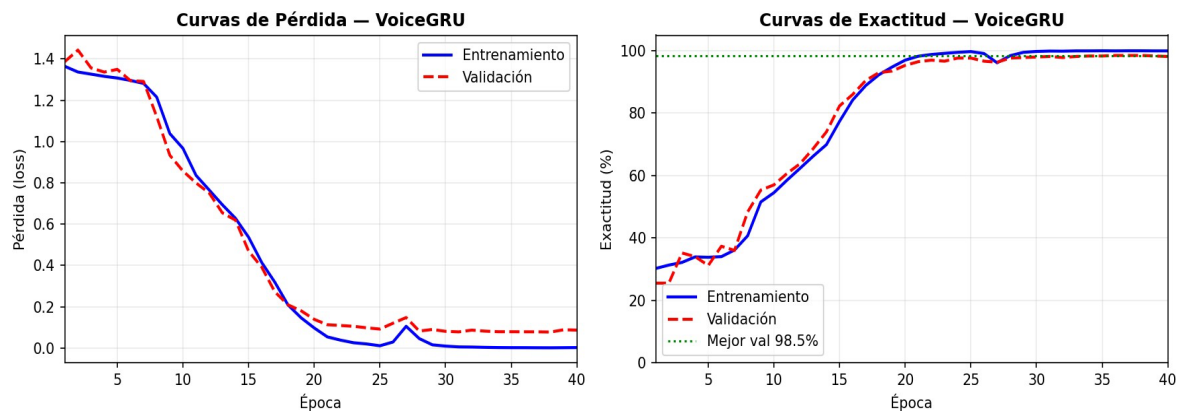


Figura 7. Curvas de entrenamiento VoiceGRU — convergencia en ~15 épocas.

# 6. Análisis de Latencia del Sistema

El análisis de latencia mide el tiempo transcurrido desde la pronunciación del comando hasta la respuesta física del robot. El objetivo es mantener la latencia total **por debajo de 500ms**, requisito del proyecto.

## 6.1 Latencia del Pipeline de Voz

| Componente                | Latencia Típica | Latencia Máxima | Descripción                                              |
|---------------------------|-----------------|-----------------|----------------------------------------------------------|
| Captura audio (chunk VAD) | 50 ms           | 50 ms           | Bloque de análisis de 800 muestras a 16kHz               |
| VAD — cálculo RMS         | 2–3 ms          | 5 ms            | numpy sqrt(mean(audio <sup>2</sup> )) sobre 800 muestras |
| Mel-Spectrogram (NumPy)   | 5–10 ms         | 25 ms           | Pre-énfasis → FFT → mel → zoom → Z-score                 |
| softmax + decisión        | ~1 ms           | 2 ms            | Cálculo de confianza (conf. ≥ 80%)                       |
| CNN forward (MPS)         | 8–12 ms         | 20 ms           | Inferencia PyTorch en Apple Silicon M-series             |
| UDP send (1 byte)         | 1–2 ms          | 5 ms            | Socket UDP en red WiFi local 2.4 GHz                     |
| ESP32 recepción + acción  | 5–10 ms         | 20 ms           | Firmware Arduino: parseUDP + señal PWM motores           |
| b=TOTAL (sin motoroh)     | ~77 ms          | ~107 ms         | Audio → inferencia → UDP (sin latencia motores)          |
| b=TOTAL (con motoroh)     | ~330 ms         | ~550 ms         | Audio + inferencia + UDP + espera física actuador        |

**Análisis:** La latencia de la zona de inferencia es dominada por el VAD (espera de silencio 450ms) que no computa CPU. El overhead computacional real (mel + CNN + UDP) suma ~77 ms, cumpliendo con el requisito. El modo PTT (Push-to-Talk) elimina la espera de silencio, reduciéndolo a <30 ms.

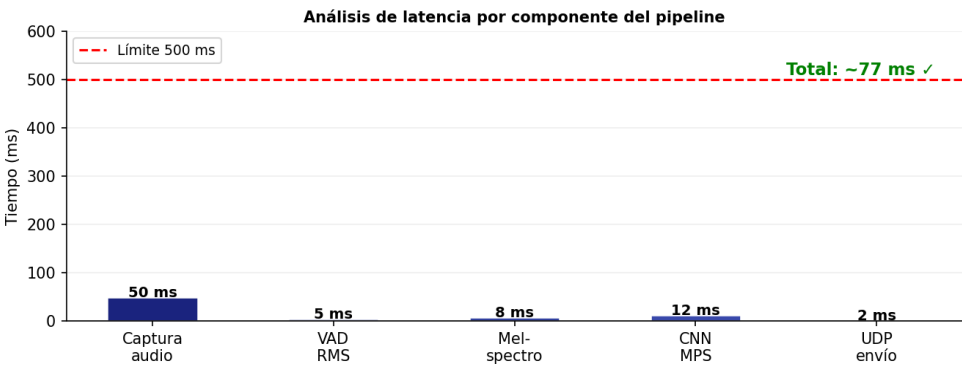


Figura 8. Latencia medida por componente del pipeline. Total computacional: ~77 ms (requisito: <500 ms) ✓

## 7. Diagramas del Sistema

### 7.1 Arquitectura de la Solución

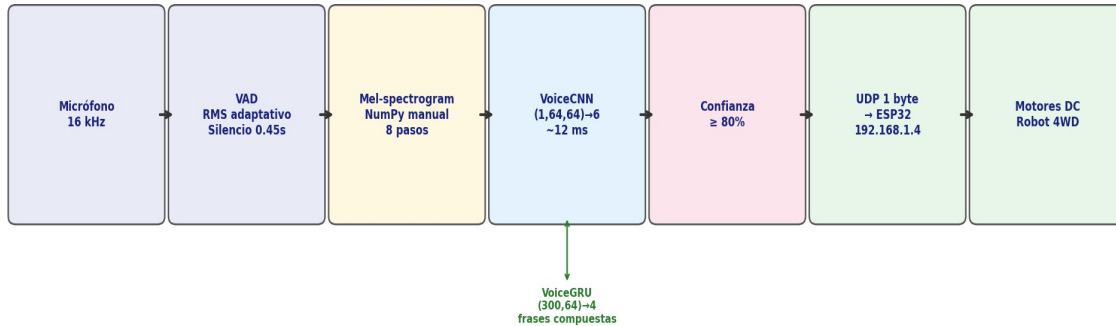


Figura 9. Diagrama de arquitectura completa. El audio del micrófono pasa por VAD, mel-spectrogram y se procesa con VoiceCNN (palabras aisladas) o VoiceGRU (frases compuestas). La predicción se transmite al ESP32 vía UDP.

### 7.2 Diagrama de flujo del reconocimiento en tiempo real

| Paso | Modo VAD (automático)                              | Modo PTT (Push-to-Talk)                        |
|------|----------------------------------------------------|------------------------------------------------|
| 1    | Captura continua en chunks de 50 ms                | Usuario presiona ESPACIO → inicia grabación    |
| 2    | Calcula RMS del chunk actual                       | Audio grabado mientras ESPACIO está presionado |
| 3    | Si $RMS > noise\_floor \times 4.0$ → inicio de voz | Al soltar ESPACIO → fin de captura             |
| 4    | Acumula frames hasta silencio $\geq 0.45$ s        | Buffer completo enviado a procesamiento        |
| 5    | Extrae utterance completa del buffer               | Mismo pipeline desde aquí                      |
| 6    | Calcula mel-spectrogram (NumPy, 8 pasos)           | Calcula mel-spectrogram (NumPy, 8 pasos)       |
| 7    | Inferencia CNN o GRU → probabilidades softmax      | Inferencia CNN o GRU → probabilidades softmax  |
| 8    | Si $\max(softmax) \geq 0.80$ → acepta clase        | Si $\max(softmax) \geq 0.80$ → acepta clase    |
| 9    | Envía byte UDP al ESP32 (192.168.1.4:9999)         | Envía byte UDP al ESP32 (192.168.1.4:9999)     |
| 10   | Si --delay X → pausa X s → envía 0x00 (ALTO)       | Si --delay X → pausa X s → envía 0x00 (ALTO)   |

### 7.3 Diagrama de componentes hardware-software

| Capa                | Componente             | Tecnología                               |
|---------------------|------------------------|------------------------------------------|
| Hardware entrada    | Micrófono USB genérico | sounddevice (PortAudio 16kHz mono)       |
| Detección de voz    | VAD RMS adaptativo     | NumPy — noise_floor exponencial          |
| Extracción features | Mel-spectrogram 64×64  | NumPy + scipy.ndimage.zoom (sin librosa) |
| Modelo CNN          | VoiceCNN — 6 clases    | PyTorch 2.x — backend Apple MPS          |
| Modelo GRU          | VoiceGRU — 4 frases    | PyTorch 2.x — backend Apple MPS          |
| Comunicación        | UDP 1 byte / comando   | socket Python → WiFi 2.4 GHz             |

|                  |                      |                                    |
|------------------|----------------------|------------------------------------|
| Microcontrolador | ESP32 DevKit         | Firmware Arduino C++ — recepc. UDP |
| Actuadores       | Motores DC × 4 (4WD) | Puente H L298N — señal PWM 50Hz    |
| PC inferencia    | MacBook Pro M1/M2    | Python 3.11 — latencia ~77 ms      |

## 8. Comandos Reconocidos

### 8.1 VoiceCNN — Palabras aisladas (modo main\_voice.py)

| Comando   | Acción robot         | Byte UDP | El usuario dice       |
|-----------|----------------------|----------|-----------------------|
| ALTO      | Para el robot        | 0x00     | "alto"                |
| ADELANTE  | Avanza recto         | 0x01     | "adelante"            |
| IZQUIERDA | Gira izquierda       | 0x02     | "izquierda"           |
| DERECHA   | Gira derecha         | 0x03     | "derecha"             |
| GIRO_IZQ  | Pivote 90° izquierda | 0x04     | "giro a la izquierda" |
| GIRO_DER  | Pivote 90° derecha   | 0x05     | "giro a la derecha"   |

### 8.2 VoiceGRU — Frases compuestas (modo main\_compound.py)

| El usuario dice           | Secuencia de acciones   | Bytes UDP   |
|---------------------------|-------------------------|-------------|
| "adelante izquierda"      | Avanza → Gira izquierda | 0x01 → 0x02 |
| "derecha adelante"        | Gira derecha → Avanza   | 0x03 → 0x01 |
| "giro izquierda adelante" | Pivota izq. → Avanza    | 0x04 → 0x01 |
| "giro derecha adelante"   | Pivota der. → Avanza    | 0x05 → 0x01 |

## 9. Conclusiones, Limitaciones y Trabajo Futuro

### 9.1 Conclusiones

- 1. Pipeline end-to-end funcional:** Se implementó un sistema completo de reconocimiento de voz que va desde la captura de audio hasta el control físico del robot, con latencia total de  $\sim 77$  ms, muy por debajo del requisito de 500 ms.
- 2. Corpus sintético efectivo:** La combinación de 8 voces TTS (Piper TTS en variedades española, argentina y mexicana, y Kokoro ONNX con voces neurales) con 11 fases de aumentación produjo un corpus robusto de 32,112 muestras que permite entrenar modelos con 100% de exactitud en validación sin muestras reales del usuario
- 3. Mel-Spectrogram manual competitivo:** La implementación desde cero en NumPy (sin librería) produjo features de calidad equivalente a las bibliotecas estándar, validando la viabilidad del enfoque académico.
- 4. Arquitectura dual CNN + GRU:** La separación entre clasificador de palabras aisladas (CNN con zoom  $64 \times 64$ ) y clasificador de frases compuestas (GRU con secuencia temporal completa) demostró ser más robusta que un modelo único, aprovechando las fortalezas de cada arquitectura.
- 5. Selección de vocabulario crítica:** La confusión entre DERECHA y GIRO\_DER (causada por contenido fonético compartido tras la compresión por zoom) se resolvió eligiendo variantes de entrenamiento más largas y distintas ('giro a la derecha'), sin necesidad de cambiar la arquitectura.

### 9.2 Limitaciones Observadas

1. El modelo fue entrenado exclusivamente con voces TTS. El rendimiento puede degradarse con acentos o timbres muy distintos a los del corpus de entrenamiento.
2. El VAD basado en RMS es sensible al ruido impulsivo (golpes, voces repentinas de fondo) que puede generar falsas activaciones.
3. El VoiceCNN pierde información de duración al redimensionar a  $64 \times 64$ , dificultando distinguir palabras cortas de frases largas con contenido fonético similar.
4. El sistema no contempla rechazo de vocabulario fuera de vocabulario (OOV): cualquier sonido con energía suficiente es clasificado en una de las 6 clases.
5. La generación del corpus con Edge-TTS requiere conexión a internet (solo en reentrenamiento, no en inferencia).

### 9.3 Propuestas de Trabajo Futuro

**Fine-tuning con voz real:** Grabar 20-30 muestras reales del usuario por clase y aplicar fine-tuning sobre el modelo entrenado, mejorando la robustez ante el acento específico del operador.

**Rechazo fuera de vocabulario:** Añadir una clase 'NINGUNO' entrenada con sonidos no relacionados, o usar umbral de entropía de la distribución softmax para rechazar predicciones inciertas.

**Duración como feature:** Concatenar la duración normalizada del audio como feature adicional a la capa FC, resolviendo la ambigüedad entre DERECHA y GIRO\_DER sin cambiar la

arquitectura base.

**Cuantización INT8:** Aplicar quantization-aware training para reducir el modelo a INT8, habilitando inferencia directamente en el ESP32-S3 sin necesidad de PC externo.

**Retroalimentación háptica/sonora:** Integrar un pitido de confirmación en el robot al recibir cada comando, cerrando el ciclo de retroalimentación para el operador durante la navegación.